

TrustBallot — Technical Requirements Document (TRD)

Version 1.1

Revision note: adds Phase 0 kiosk registration, fixes module ownership to match PRD v1.2, specifies threshold decryption parameter, coarsens timestamp bucket, switches ZKP scheme to avoid trusted-setup risk, adds error-handling and storage-growth notes.

0. System Architecture Overview



1. Technology Stack

Layer	Tech
Kiosk client (Wasm crypto)	Rust → WebAssembly, or JS + WebAssembly ZKP lib
ZKP circuit	Circom / arkworks, PLONK (universal setup — avoids per-circuit trusted-setup ceremony risk of Groth16)
Post-quantum signature	CRYSTALS-Dilithium (liboqs bindings)

Layer	Tech
Blockchain	Hyperledger Fabric or Besu (permissioned/consortium), 4 nodes on separate VM/container instances (not co-located — required for genuine BFT fault-tolerance testing)
Consensus	Custom BFT (PBFT-style), quorum = $\lceil (n+f+1)/2 \rceil$
Homomorphic tally	Paillier or ElGamal-based additive HE (python-paillier for prototype). Classical security only — not post-quantum. Documented gap: signatures are PQ-safe, ballot secrecy is not.
Backend/API	Node.js/Express or FastAPI
Frontend kiosk UI	React (offline-first PWA)
Node networking	gRPC or libp2p between 4 consortium nodes
Face-match	Stubbed (mock pass/fail) for MVP — no model/library selected; real implementation out of scope

2. Data Models

Voter Credential (client-side only, never transmitted raw): `{ epic_hash, constituency_id, merkle_proof }` *Merkle tree is a frozen snapshot of the registered voter pool as of election start — no mid-election updates in MVP (documented boundary).*

Vote Payload (on-chain): `{ ciphertext_vote, zkp_proof, nullifier, dilithium_signature, election_id, timestamp_bucket }` *granularity fixed at per-election-day (not per-hour/minute) — finer granularity would partially defeat the shuffled-batch metadata obfuscation.*

Nullifier Ledger (per election): `{ election_id: [spent_nullifiers] }` — append-only, checked $O(1)$ via hash set/Merkle set. *Storage growth is unbounded per election; no pruning/archival strategy defined — acceptable for demo scale, flagged as a Future Feature dependency for any real deployment.*

Kiosk Registry (new, Phase 0): `{ kiosk_id, kiosk_pubkey, eci_signature }` — ECI signs each kiosk's pubkey before election open; contract rejects payloads from unregistered kiosks.

3. Cryptographic Protocols

ZKP circuit constraints: (1) credential ownership, (2) Merkle membership in registered voter pool (frozen snapshot), (3) one-hot candidate encoding, (4) correct nullifier derivation from

`(private_credential, election_id)`.

Nullifier: `Hash(private_credential || election_id)` — deterministic, unlinkable.

Signing: Dilithium keypair per kiosk session; signs the encrypted payload, separate from ZKP privacy layer.

EPIC/QR signature verification: kiosk verifies the scanned credential's signature against ECI's public key before proceeding — this step is cryptographic and owned by Anushka, distinct from the scan/camera UI (Arnav).

Homomorphic tally: additive HE, $\text{Enc}(v1) + \text{Enc}(v2) = \text{Enc}(v1+v2)$; final decryption requires **threshold t=3-of-4** consortium key shares (Shamir's Secret Sharing), matching the BFT quorum so tally isn't blocked by the same single node outage the consensus layer already tolerates.

4. Smart Contract Logic (pseudocode)



```
function registerKiosk(kiosk_id, pubkey, eci_signature):
  require(verifyECISignature(pubkey, eci_signature))
  kioskRegistry.add(kiosk_id, pubkey)

function submitVote(payload):
  require(kioskRegistry.has(payload.kiosk_id))
  require(verifyDilithium(payload.signature))
  require(verifyZKP(payload.zkp_proof))
  if NullifierSet.has(payload.nullifier):
    nullifierSet.add(payload.nullifier)
    ledger.append(payload.ciphertext_vote)
    emit VoteAccepted(payload.election_id)
  else:
    emit VoteRejected(payload.election_id, reason="duplicate_nullifier")
  # triggers voter-facing error/retry flow (see Design Spec)
```

5. Consensus & Fault Tolerance

4 nodes: ECI, Judiciary nominee, independent auditor, civil-society rep — **deployed on separate VM/container instances**, not one shared VM (fixes a demo-integrity gap: a single-VM failure would previously kill all 4 "nodes" at once, invalidating the resilience test).

BFT quorum tolerates 1 faulty/offline node without halting.

Block finalization requires quorum-signed commitments.

6. Interoperability

EVM booths emit a lightweight signal on manual voter marking → generates nullifier via same hash function.

IBC-style batch sync every ~2 hrs in low-connectivity zones.

Collision on sync → auto-flag "Disputed Vote" for **manual, procedural** review — no biometric data is retained to cryptographically resolve disputes (face-match data is purged post-scan by design); resolution requires in-person re-authentication, documented as an explicit limitation rather than an automated fix.

7. Security Requirements

No raw biometric/EPIC data leaves kiosk (purged post-scan).

Metadata obfuscation: batch size 50–100 votes, shuffled before submission, **OR flushed after a 2-hour timeout** (mitigates the low-turnout-kiosk problem where a batch may never reach 50 votes; documented tradeoff — small forced batches have a weaker anonymity set).

Post-quantum signature scheme mandatory for long-term integrity (does not extend to ballot secrecy — see §1).

Hardware attestation required for kiosk boot (documented dependency, not built in MVP).

Kiosk pubkey must be pre-registered via Phase 0 before any vote payload is accepted.

8. Non-Functional Requirements

ZKP generation: target $\leq 150\text{ms}$ client-side.

Smart contract verification: $\leq 10\text{ms}$ combined (sig + ZKP + nullifier check).

System must degrade gracefully to offline batch mode.

Voter-facing error state must render within 1s of a contract rejection, with a defined retry path (see Design Spec §3a).

9. Module Ownership

(All cryptography consolidated under Anushka, per team decision — this table now matches PRD v1.2 exactly.)

Anushka: ZKP circuit (PLONK), nullifier logic, smart contract, HE tally + threshold decryption, Dilithium integration, EPIC/QR signature verification, kiosk pubkey registration (crypto half).

Shashwat: BFT node network (separate instances), IBC/EVM sync, disputed-vote backend logic, kiosk registration infra (registration half), DevOps.

Arnav: Kiosk client UI (scan, PIN, candidate grid + pagination, error/retry states), face-match stub, offline-first PWA shell, metadata obfuscation middleware (batching + 2-hr flush), API integration layer, dashboard.